

Counters

- Asynchronous VS Synchronous counter
- Up/Down/Load counter
- Modulus counter
- BCD & Multi Decode counter
- counter as Timer

Q Asynchronous counter

clock of consecutive f.f. is output of previous f.f.

T	$Q(t+1)$
0	$Q(t)$
1	$\overline{Q(t)}$

→ constructing T.F.F.

module TFF = (

);

localparam delay = 2;

reg Q-reg;

wire Q-next;

// register logic

always @ (tvc clk, -ve reset.n)

begin

if (~reset.n)

Q-reg = 0;

else

#delay Q-reg <= Q-next;

end

// next state logic

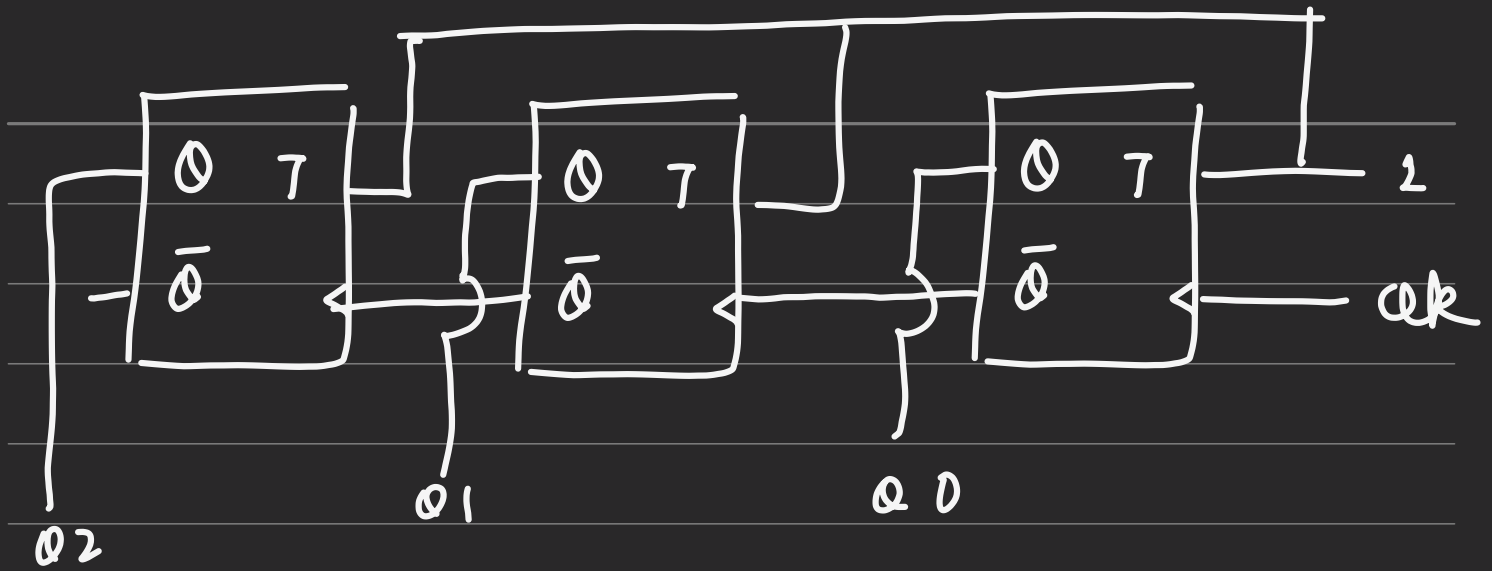
assign Q-next = T ? ~Q-reg : Q-reg;

// output logic

assign Q = Q-reg;

endmodule

↳ Generic Async UP Counter



module

(parameter N =)

(

) ;

generate i ;

generate

for (i = 0 ; i < N ; i = i + 1

begin : tff chain

wire tff_clk = (c == 0) ? clk :
Q[i-1];

```
T_tff_inst0(  
    T(1'b1),  
    .clk(tff_clk),  
    .reset_n(reset_n),  
    .Q(Q[i]).  
);  
end;  
endgen -  
endmodule.
```

issue with async counter: glitches
due to $< 2G$ delay

↳ Test bench

```
module _____ ();  
    // declaring port  
    reg clk, reset_n;  
    wire [3:0] Q;
```

```
    // Instantiate module under test  
    async-up-counter demo(  
        .clk(clk),  
        .reset_n(reset_n),  
        .Q(Q)
```

—);

// timer

initial

200 \$ stop

// Generating Clk signal

local param T = 20;

always

begin

clk = 1b'0;

(T/2);

clk = 1b'1;

(T/2);

end.

// quick reset for 2ns

initial

begin

reset_n = 1'b0;

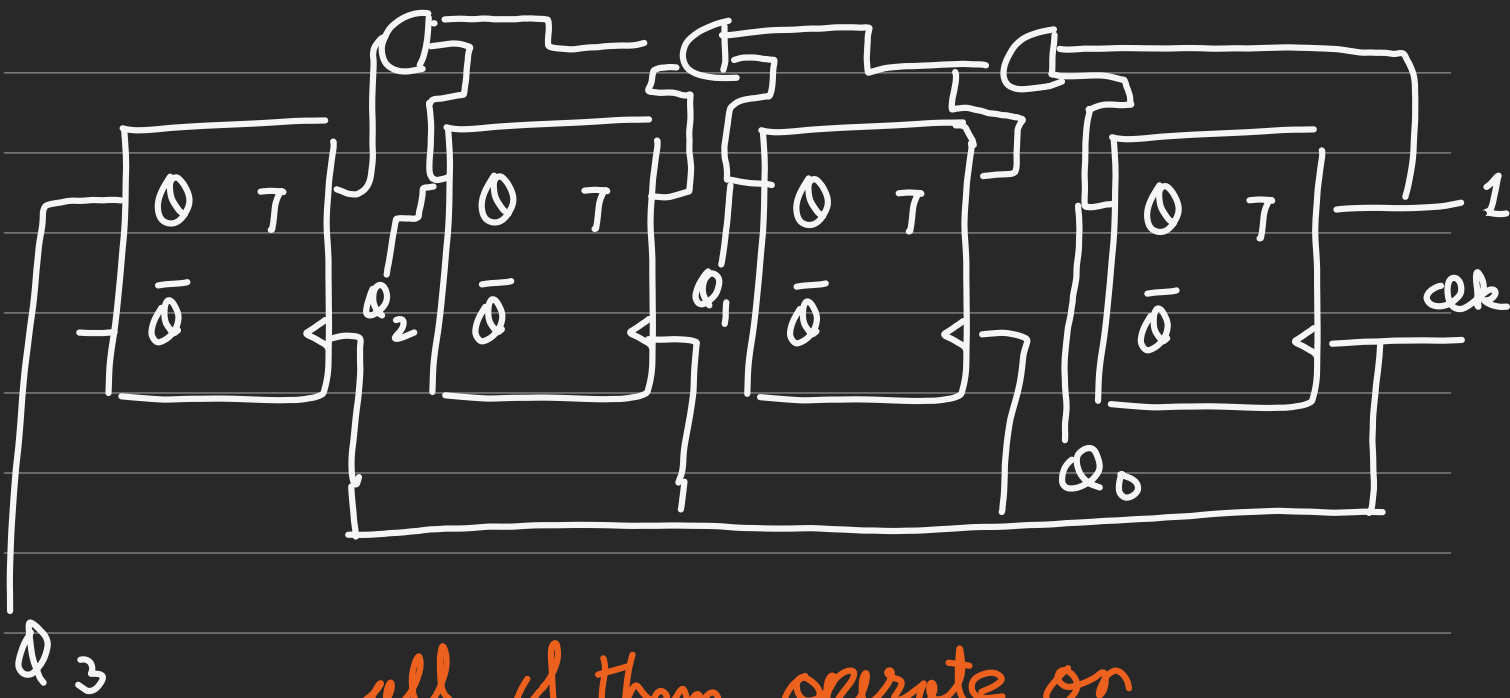
2 reset_n = 1'b1;

end

Asyn counter aka ripple counter.
& due to this 2G delay, they
can't be used for fast

units.

② Synchronous Counter



all of them operate on
same clock

↳ code

```
module synch-up-counter(  
    _____  
    _____  
    _____  
);
```

```
wire [3:0] Q_next;
```

assign Q_next[0] = 1'b1;

```
TFF FF0(  
    .clk(—)  
    .T(Q_next[0])  
    .reset_n(—)  
    .Q(Q[0]));
```

assign Q_next[1] = Q[0] & Q_next[0];

```
TFF FF1(  
    .clk(clk)  
    .T(Q_next[1])  
    .reset_n(—)  
    .Q(Q[1]);
```

assign Q_next[2] = Q[1] & Q_next[1];

|

endm

